ROOTCAUSE SERVER

05 · Referencia técnica

Documento: 05-technical-reference.md **Versión analizada:** 0.2.0 **Commit:** 2efedb2 **Fecha del análisis:** 2026-08-26

CONTENIDO

- Constantes del sistema
  - Protocolo y dominio (rootcause-core)
  - Servidor (rootcause-server)
  - Agente (rootcause-agent)
- Variables de entorno
  - Servidor
  - Agente
- Comandos
- Tipos de datos del dominio
  - Enumeraciones
  - Estructuras que viajan por el cable
- Catálogo de reglas
- Umbrales por omisión
- Funciones clave
  - DetectionEngine::analyze
  - DetectionPolicy::exposure_severity
  - BindScope::classify
  - posture::compute
  - auth::require_auth
  - api::ingest_telemetry
  - storage::Database::upsert_incident
  - storage::Database::auto_resolve
  - defense::Perimeter::check
  - probe::run
  - validate_transport
  - stable_agent_id
- Endpoints
- Códigos de error
- Métricas de Prometheus
- Estados
- Eventos registrados
- Archivos de configuración

> Catálogo de consulta. Constantes, tipos, funciones, variables de entorno, opciones de línea de comandos, endpoints, códigos de error, métricas y archivos de configuración, con su firma real y su ubicación exacta.

Constantes del sistema

Protocolo y dominio (`rootcause-core`)

Constante	Valor	Archivo	Para qué
`PROTOCOL_VERSION`	`"1.1"`	`lib.rs:30`	Versión del contrato de cable que emite este build
`SUPPORTED_PROTOCOL_VERSIONS`	`["1.0", "1.1"]`	`lib.rs:33`	Versiones que el servidor todavía ingiere
`RULES`	18 entradas `RuleInfo`	`detect/mod.rs:47`	Catálogo publicado de reglas
`PORT_CATALOG`	48 entradas `PortProfile`	`security.rs:157`	Puertos cuya exposición pública cambia el riesgo
`ACCESS_CRITICAL`	10 marcadores de ruta	`detect/integrity.rs:15`	Archivos cuya modificación cambia quién entra

Servidor (`rootcause-server`)

Constante	Valor	Archivo	Para qué
`HISTORY_WINDOW`	`12`	`api.rs:37`	Muestras de historial entregadas a los detectores por ingesta
`STALE_AFTER_SECONDS`	`120`	`storage.rs:51`	A partir de aquí un activo está `stale`
`OFFLINE_AFTER_SECONDS`	`600`	`storage.rs:52`	A partir de aquí un activo está `offline`
`MAX_TRACKED_CLIENTS`	`20_000`	`defense.rs:21`	Direcciones vigiladas antes de desalojar la más antigua
`TICK`	`30 s`	`watchdog.rs:20`	Cadencia del vigilante
`PURGE_EVERY_TICKS`	`120`	`watchdog.rs:22`	Retención cada 120 ticks ≈ 1 hora
`PERIMETER_IDLE`	`900 s`	`watchdog.rs:24`	Inactividad tras la que se olvida una entrada del perímetro
`CONTENT_SECURITY_POLICY`	cadena CSP	`headers.rs:20`	Política aplicada a toda respuesta
`SECURITY_HEADERS`	8 pares	`headers.rs:31`	Cabeceras estáticas de toda respuesta
`STRICT_TRANSPORT_SECURITY`	`max-age=31536000; includeSubDomains`	`headers.rs:45`	Solo si la petición llegó por TLS
`UNKNOWN_CLIENT`	`0.0.0.0`	`auth.rs:23`	Marcador estable cuando la conexión no trae información de par

Agente (`rootcause-agent`)

Constante	Valor	Archivo	Para qué
`MAX_BACKOFF`	`300 s`	`main.rs:29`	Techo del reintento exponencial

Constante	Valor	Archivo	Para qué
`ALLOWED_PROGRAMS`	11 programas	`probe.rs:17`	Lista blanca de comandos de solo lectura
`MAX_LISTENERS`	`256`	`net.rs:21`	Tope de sockets en escucha reportados por ciclo
`MAX_PEERS`	`512`	`net.rs:22`	Tope de pares conectados reportados por ciclo
`MAX_EVENTS`	`200`	`authlog.rs:18`	Tope de filas de autenticación agregadas por ciclo
`MAX_FILE_BYTES`	`8 MiB`	`integrity.rs:15`	Tamaño máximo de archivo vigilado
`PROBE_TIMEOUT`	`5 s` / `6 s` / `8 s`	`net.rs`, `baseline.rs`, `authlog.rs`	Tiempo límite por familia de sonda
`LINUX_DEFAULTS`, `WINDOWS_DEFAULTS`, `MACOS_DEFAULTS`	listas de rutas	`integrity.rs:18–35`	Lista de vigilancia por omisión de cada plataforma

Variables de entorno

Servidor

Variable	Opción equivalente	Tipo	Por omisión	Rango válido
`ROOTCAUSE_BIND`	`--bind`	`SocketAddr`	`127.0.0.1:8080`	Cualquier dirección válida
`ROOTCAUSE_DATABASE_URL`	`--database-url`	cadena	`sqlite://rootcause.db`	Debe empezar por `sqlite:`
`ROOTCAUSE_API_TOKEN`	`--api-token`	cadena	*(sin valor)*	≥ 32 caracteres, sin espacios envolventes
`ROOTCAUSE_INSECURE_DEV_MODE`	`--insecure-dev-mode`	booleano	`false`	Solo con enlace en loopback
`ROOTCAUSE_JSON_LOGS`	`--json-logs`	booleano	`false`	—
`ROOTCAUSE_RATE_LIMIT_PER_MINUTE`	`--rate-limit-per-minute`	`u32`	`600`	10 – 100 000
`ROOTCAUSE_LOCKOUT_THRESHOLD`	`--lockout-threshold`	`u32`	`10`	3 – 1 000
`ROOTCAUSE_LOCKOUT_SECONDS`	`--lockout-seconds`	`u64`	`300`	30 – 86 400
`ROOTCAUSE_RETENTION_DAYS`	`--retention-days`	`u32`	`30`	1 – 3 650
`ROOTCAUSE_AGENT_INTERVAL_SECONDS`	`--agent-interval-seconds`	`u64`	`30`	5 – 3 600
`ROOTCAUSE_TRUST_FORWARDED_FOR`	`--trust-forwarded-for`	booleano	`false`	Incompatible con enlace en loopback
`ROOTCAUSE_POLICY_FILE`	`--policy-file`	ruta	*(sin valor)*	JSON `DetectionPolicy` válido
`ROOTCAUSE_MAX_BODY_KIB`	`--max-body-kib`	`u64`	`1024`	16 – 16 384
`ROOTCAUSE_LOG`	—	filtro `EnvFilter`	`rootcause_server=info,tower_http=warn`	Sintaxis de `tracing-subscriber`

Agente

Variable	Opción equivalente	Tipo	Por omisión	Rango válido
`ROOTCAUSE_SERVER_URL`	`--server-url`	URL	`http://127.0.0.1:8080`	Esquema `http` o `https`
`ROOTCAUSE_API_TOKEN`	`--api-token`	cadena	**obligatorio**	≥ 32 caracteres
`ROOTCAUSE_AGENT_INTERVAL_SECONDS`	`--interval-seconds`	`u64`	`30`	5 – 3 600
`ROOTCAUSE_AGENT_METRICS_ONLY`	`--metrics-only`	booleano	`false`	—
`ROOTCAUSE_AGENT_AUTH_WINDOW_MINUTES`	`--auth-window-minutes`	`u32`	`15`	1 – 1 440
`ROOTCAUSE_WATCH_FILES`	`--watch-list`	lista separada por coma	*(sin valor)*	Rutas; reemplaza las de la plataforma
`ROOTCAUSE_LOG`	—	filtro `EnvFilter`	`rootcause_agent=info`	—

Opciones del agente **sin** variable de entorno: `--once`, `--dry-run`, `--allow-insecure-http`, `--watch-file` (repetible), `--label clave=valor` (repetible).

Comandos

```
rootcause-server serve  [--bind ...] [--database-url ...] [--api-token ...]
                        [--insecure-dev-mode] [--json-logs]
                        [--rate-limit-per-minute N] [--lockout-threshold N]
                        [--lockout-seconds N] [--retention-days N]
                        [--agent-interval-seconds N] [--trust-forwarded-for]
                        [--policy-file RUTA] [--max-body-kib N]
rootcause-server token                  # 64 caracteres hexadecimales
rootcause-server rules                  # catálogo de 18 reglas
rootcause-server policy [--file RUTA]   # imprime o valida una política

rootcause-agent [--server-url URL] [--api-token TOKEN] [--interval-seconds N]
                [--once] [--dry-run] [--metrics-only] [--allow-insecure-http]
                [--auth-window-minutes N] [--watch-file RUTA]...
                [--watch-list a,b,c] [--label clave=valor]...
```

Tipos de datos del dominio

Enumeraciones

Tipo	Valores serializados	Archivo
`Platform`	`windows`, `linux`, `macos`, `unknown`	`models.rs:14`
`AssetRole`	`edge-server`, `internal-server`, `database-server`, `workstation`	`models.rs:56`
`Severity`	`info`, `low`, `medium`, `high`, `critical`	`models.rs:177`

Tipo	Valores serializados	Archivo
`Category`	`resource`, `exposure`, `intrusion`, `integrity`, `hygiene`, `availability`	`models.rs:231`
`IncidentStatus`	`open`, `acknowledged`, `resolved`	`models.rs:295`
`AssetStatus`	`online`, `stale`, `offline`	`models.rs:448`
`Protocol`	`tcp`, `udp`	`security.rs:16`
`BindScope`	`loopback`, `private`, `public`	`security.rs:36`
`PortClass`	`remote-admin`, `database`, `infrastructure`, `cleartext`, `file-share`, `mail`, `web`, `other`	`security.rs:97`
`AuthOutcome`	`failure`, `success`	`security.rs:276`
`StepKind`	`inspect`, `contain`, `remediate`	`runbook.rs:16`
`Decision` (interno)	`Allow`, `RateLimited{..}`, `LockedOut{..}`	`defense.rs:26`
`ProbeError` (interno)	`NotAllowed`, `Missing`, `Failed(i32)`, `TimedOut`	`probe.rs:40`

Estructuras que viajan por el cable

Tipo	Campos	Dirección
`AssetRegistration`	`agent_id`, `hostname`, `platform`, `os_version?`, `kernel_version?`, `architecture`, `agent_version`, `labels`	Agente → Servidor
`MetricSample`	`agent_id`, `observed_at`, `cpu_percent`, `memory_percent`, `disk_percent`, `uptime_seconds`, `load_average?`, `network_rx_bytes`, `network_tx_bytes`, `disk_free_bytes?`, `process_count?`	Agente → Servidor
`SecuritySignals`	`listeners`, `peers`, `auth_events`, `watched_files`, `firewall?`, `pending_security_updates?`, `collection_gaps`	Agente → Servidor
`TelemetryEnvelope`	`protocol_version`, `asset?`, `sample`, `security?`	Agente → Servidor
`IngestResponse`	`accepted`, `incidents_touched`, `warnings`	Servidor → Agente
`Incident`	`id`, `fingerprint`, `asset_id`, `title`, `summary`, `severity`, `category`, `status`, `root_cause`, `confidence`, `first_seen`, `last_seen`, `occurrences`, `evidence`, `recommended_actions`, `runbook`, `techniques`	Servidor → Consola
`StatusResponse`	`service`, `version`, `protocol_version`, `uptime_seconds`, `assets_total`, `assets_online`, `open_incidents`, `critical_incidents`, `exposed_services`, `blocked_sources`, `detectors`, `posture?`, `hardening`	Servidor → Consola
`HardeningStatus`	`authentication`, `bind_is_loopback`, `rate_limit_per_minute`, `lockout_threshold`, `retention_days`	Servidor → Consola

Los campos con `?` llevan `#[serde(skip_serializing_if = "Option::is_none")]`: no aparecen en el JSON cuando están vacíos.

Catálogo de reglas

Id	Categoría	Techo	Técnicas ATT&CK	Pregunta que responde
`exposure.service.public`	exposure	critical	T1190	¿Hay un servicio escuchando en una dirección que no es loopback?
`exposure.cleartext.protocol`	exposure	high	T1040, T1190	¿Se publican credenciales en claro por Telnet, FTP, LDAP o POP3?
`intrusion.auth.bruteforce`	intrusion	critical	T1110.001	¿Un mismo origen acumula intentos fallidos sobre este servidor?
`intrusion.auth.spray`	intrusion	high	T1110.003	¿Un origen prueba muchos nombres de usuario distintos?
`intrusion.auth.distributed`	intrusion	high	T1110	¿Muchos orígenes distintos fallan contra el mismo servidor?
`intrusion.auth.success_after_burst`	intrusion	critical	T1110, T1078	¿Un origen que fallaba repetidamente consiguió entrar?
`intrusion.network.fanin`	intrusion	high	T1046, T1595	¿Demasiadas direcciones distintas tocan el host a la vez?
`intrusion.egress.anomaly`	intrusion	high	T1041	¿El tráfico de salida se disparó respecto de su propia línea base?
`integrity.file.changed`	integrity	high	T1543, T1098	¿Cambió un archivo de configuración que sostiene la seguridad?
`integrity.file.permissions`	integrity	high	T1222	¿Un archivo sensible quedó escribible por cualquier usuario?
`hygiene.firewall.disabled`	hygiene	high	T1562.004	¿El servidor depende solo del perímetro para filtrar tráfico?
`hygiene.updates.pending`	hygiene	high	T1190	¿Hay parches de seguridad publicados y no aplicados?
`hygiene.clock.skew`	hygiene	medium	T1070.006	¿La hora del servidor permite correlacionar sus registros?
`availability.agent.silence`	availability	high	T1562.001	¿El agente calló sin que nadie lo detuviera de forma planificada?
`resource.cpu.saturation`	resource	critical	T1496	¿La CPU lleva varias muestras seguidas al límite?
`resource.memory.pressure`	resource	critical	T1499	¿La memoria comprometida puede provocar paginación o OOM?
`resource.disk.capacity`	resource	critical	T1499	¿Queda margen de disco para operar y registrar?
`resource.disk.runway`	resource	high	T1499	¿Cuántas horas faltan para quedarse sin disco al ritmo actual?

Dos invariantes que CI verifica: todo id está **namespaced por su categoría**, y toda regla declara pregunta y al menos una técnica.

Umbrales por omisión

Grupo	Campo	Valor
`resource`	`cpu_high` / `cpu_critical`	`90.0` / `98.0`

Grupo	Campo	Valor
`resource`	`cpu_sustained_samples`	`3`
`resource`	`memory_high` / `memory_critical`	`88.0` / `96.0`
`resource`	`disk_high` / `disk_critical`	`85.0` / `95.0`
`resource`	`disk_runway_hours`	`48.0`
`intrusion`	`failures_per_source_high` / `_critical`	`20` / `100`
`intrusion`	`sprayed_usernames`	`5`
`intrusion`	`distributed_sources`	`10`
`intrusion`	`peer_fanin_high` / `_critical`	`50` / `200`
`intrusion`	`egress_multiplier`	`8.0`
`intrusion`	`egress_floor_bytes_per_second`	`1_000_000.0`
`hygiene`	`pending_updates_high`	`15`
`hygiene`	`clock_skew_seconds`	`120`
`hygiene`	`silence_intervals`	`4`
(raíz)	`public_allowlist`	`[]`

Funciones clave

Formato: **firma** · propósito · parámetros · retorno · errores · efectos secundarios · quién la llama · a quién llama · riesgos al modificarla.

`DetectionEngine::analyze`

```
pub fn analyze(&self, input: &DetectionInput<'_>) -> Vec<IncidentCandidate>
```

- **Archivo:** `crates/rootcause-core/src/detect/mod.rs:263`
- **Propósito:** ejecutar las cinco familias de reglas sobre un sobre de telemetría y devolver los hallazgos ordenados de mayor a menor severidad.
- **Parámetros:** `input` — registro del activo, muestra, superficie opcional, historial (más antiguo primero, sin incluir la muestra actual), línea base de huellas de archivos e instante de recepción del servidor.
- **Retorno:** vector de candidatos ya fusionados por huella. Vacío si no hay nada que reportar.
- **Errores:** ninguno. La función no falla; una política inválida se rechaza antes, en `DetectionEngine::new`.
- **Efectos secundarios:** ninguno. Sin E/S, sin reloj propio, sin aleatoriedad.
- **La llama:** `api::ingest_telemetry`.
- **Llama a:** `exposure::detect`, `intrusion::detect`, `integrity::detect`, `hygiene::detect`, `resource::detect`, `merge_by_fingerprint`.
- **Riesgos al modificarla:** el orden de fusión decide qué evidencia sobrevive cuando dos candidatos comparten huella. Alterar el criterio de ordenación cambia qué hallazgo lidera la consola.

`DetectionPolicy::exposure_severity`

```
pub fn exposure_severity(&self, class: PortClass, scope: BindScope, role: AssetRole) -> Severity
```

- **Archivo:** `crates/rootcause-core/src/policy.rs:175`
- **Propósito:** decidir la severidad de un servicio expuesto. Es **la regla sobre la que descansa toda la vista de superficie**.
- **Lógica:** severidad base por clase de puerto; `Loopback` corta y devuelve `Info`; `Private` baja un escalón; un servidor de base de datos alcanzable públicamente en un puerto conocido es siempre `Critical`.
- **Retorno:** `Severity`. `Info` significa «no generar hallazgo».
- **La llama:** `detect::exposure::detect` y `api::build_exposure`.
- **Riesgos:** subir el escalón de `Private` inundaría la consola en redes internas planas; bajarlo escondería una base de datos alcanzable desde toda la oficina.

`BindScope::classify`

```
pub fn classify(address: &str) -> Self
```

- **Archivo:** `crates/rootcause-core/src/security.rs:59`
- **Propósito:** derivar el alcance real de una dirección de enlace textual.
- **Casos:** `*` y cadena vacía → `Public`; dirección no parseable → `Public`; loopback → `Loopback`; no especificada (`0.0.0.0`, `::`) → `Public`; RFC1918, link-local y CGNAT (100.64/10) → `Private`; ULA `fc00::/7` y link-local `fe80::/10` → `Private`; resto → `Public`.
- **Riesgos:** es el punto donde el producto decide fallar hacia el caso peor. Convertir lo ilegible en `Private` silenciaría hallazgos reales.

`posture::compute`

```
pub fn compute(incidents: &[Incident], signals: Option<&SecuritySignals>,
               computed_at: DateTime<Utc>) -> PostureScore
```

- **Archivo:** `crates/rootcause-core/src/posture.rs:60`
- **Propósito:** resumir la defensa en un número de 0 a 100 **con su lista de superficies no inspeccionadas**.
- **Lógica:** por categoría resta 45/25/10/4/0 puntos según severidad; pondera las dimensiones 5/5/3/3/2/1; aplica un techo (55 crítico, 74 alto, 89 medio) derivado del peor hallazgo abierto.
- **Errores:** ninguno; `checked_div` protege la media.
- **Riesgos:** el techo es lo que impide que un servidor con una base de datos expuesta obtenga una nota tranquilizadora. Quitarlo rompe REQ-SEC-003.

`auth::require_auth`

```
pub async fn require_auth(State(state): State<AppState>, request: Request, next: Next)
    -> Result<Response, ApiError>
```

- **Archivo:** `crates/rootcause-server/src/auth.rs:58`
- **Propósito:** puerta de entrada de toda ruta protegida.
- **Orden, que es el punto del módulo:** 1· límite de tasa, 2· bloqueo, 3· comparación del token.
- **Errores:** `429` con `Retry-After` si el perímetro rechaza; `401` si el token no coincide.
- **Efectos secundarios:** registra eventos de defensa en la base de datos; puede activar un bloqueo.

- **Riesgos:** mover la comparación del token delante del perímetro convierte el endpoint en un oráculo para quien adivina credenciales (ADR 0003).

`api::ingest_telemetry`

```
async fn ingest_telemetry(State(state): State<AppState>, Json(envelope): Json<TelemetryEnvelope>)
    -> Result<Json<IngestResponse>, ApiError>
```

- **Archivo:** `crates/rootcause-server/src/api.rs:229`
- **Validaciones, en orden:** versión de protocolo admitida → `MetricSample::validate` → ventana temporal (`observed_at` no más de 5 minutos en el futuro ni más de 24 horas en el pasado) → validación del registro → coincidencia de `agent_id` → activo existente → límites de la superficie de seguridad.
- **Errores:** `400` por protocolo, muestra, ventana, identificadores o límites; `404` si el activo no está registrado; `500` por fallo de almacenamiento.
- **Efectos secundarios:** escribe muestra, superficie, presión de autenticación, incidentes, línea base de archivos, postura y auditoría.
- **Riesgos:** es la función más larga del servidor y concentra todo el ciclo. Cualquier cambio en el orden de `auto_resolve` respecto de `upsert_incident` puede cerrar hallazgos que siguen vigentes.

`storage::Database::upsert_incident`

```
pub async fn upsert_incident(&self, candidate: IncidentCandidate, observed_at: DateTime<Utc>)
    -> anyhow::Result<Incident>
```

- **Archivo:** `crates/rootcause-server/src/storage.rs:391`
- **Propósito:** convertir un candidato en incidente nuevo, o actualizar el que ya existe con la misma huella.
- **Comportamiento en actualización:** avanza `occurrences`, refresca `last_seen`, **vuelve a abrir** el incidente aunque estuviera reconocido (reconocer no es arreglar) y conserva la severidad **máxima** vista.
- **Riesgos:** perder el `max` de severidad haría que un incidente crítico se degradase solo porque un ciclo posterior lo vio menos grave.

`storage::Database::auto_resolve`

```
pub async fn auto_resolve(&self, agent_id: Uuid, categories: &[Category],
                          still_present: &BTreeSet<String>) -> anyhow::Result<usize>
```

- **Archivo:** `crates/rootcause-server/src/storage.rs:457`
- **Propósito:** cerrar los hallazgos de ciertas categorías que el último ciclo ya no reporta, dejando constancia del motivo en la auditoría.
- **Precondición crítica:** `categories` solo debe contener familias cuya superficie se **inspeccionó de verdad** en ese ciclo. Lo decide `api::self_healing_categories`.
- **Riesgos:** ampliar la lista de categorías sin comprobar brechas reintroduce el fallo que REQ-SEC-003 existe para evitar.

`defense::Perimeter::check`

```
pub fn check(&self, address: IpAddr, now: Instant) -> Decision
```

- **Archivo:** `crates/rootcause-server/src/defense.rs:83`
- **Propósito:** consumir un espacio de la cubeta de tokens de esa dirección.
- **Retorno:** `Allow`, `RateLimited{retry_after_seconds}` o `LockedOut{retry_after_seconds}`.
- **Detalles:** el bloqueo expirado perdona el contador; la cubeta se rellena a `rate_limit_per_minute / 60` por segundo; si la tabla llega a `MAX_TRACKED_CLIENTS` se desaloja la entrada más antigua.
- **Riesgos:** el estado es solo en memoria, deliberadamente. Persistirlo convertiría un reinicio en un bloqueo heredado.

`probe::run`

```
pub async fn run(program: &str, args: &[&str], timeout: Duration) -> Result<String, ProbeError>
```

- **Archivo:** `crates/rootcause-agent/src/probe.rs:66`
- **Propósito:** ejecutar un comando de la lista blanca en solo lectura.
- **Garantías:** programa fuera de la lista → `NotAllowed` **antes** de tocar el sistema operativo; `stdin` nulo; `stderr` descartado; `kill_on_drop`; tiempo límite.
- **Riesgos:** es el único punto de ejecución externa del agente. Añadir un programa aquí amplía la superficie de todo el producto.

`validate_transport`

```
fn validate_transport(url: &Url, allow_insecure_http: bool) -> anyhow::Result<()>
```

- **Archivo:** `crates/rootcause-agent/src/main.rs:269`
- **Propósito:** negarse a poner un token *bearer* en un HTTP remoto.
- **Acepta:** `https` siempre; `http` solo hacia `localhost` o una IP de loopback, salvo `--allow-insecure-http` explícito.

`stable_agent_id`

```
pub(crate) fn stable_agent_id(hostname: &str) -> Uuid
```

- **Archivo:** `crates/rootcause-agent/src/main.rs:300`
- **Propósito:** que el mismo equipo conserve su identidad entre reinicios, sin guardar estado en disco. UUID v5 derivado del nombre de equipo.
- **Riesgo conocido:** dos equipos con el mismo `hostname` colisionan y comparten activo. Recogido en 15 · Riesgos.

Endpoints

Contrato completo en 09 · APIs e integraciones. Resumen:

Método	Ruta	Éxito	Autenticada
GET	`/healthz`	200	No
GET	`/readyz`	200 · 500	No
GET	`/api/v1/status`	200	Sí
GET	`/api/v1/rules`	200	Sí
GET	`/api/v1/policy`	200	Sí
GET	`/api/v1/assets`	200	Sí
POST	`/api/v1/assets/register`	204	Sí
GET	`/api/v1/assets/{id}`	200 · 404	Sí
POST	`/api/v1/telemetry`	200 · 400 · 404	Sí
GET	`/api/v1/incidents`	200 · 400	Sí
GET	`/api/v1/incidents/{id}`	200 · 404	Sí
POST	`/api/v1/incidents/{id}/status`	200 · 400 · 404	Sí
GET	`/api/v1/incidents/{id}/runbook`	200 · 404	Sí
GET	`/api/v1/exposure`	200	Sí
GET	`/api/v1/threats`	200	Sí
GET	`/api/v1/topology`	200	Sí
GET	`/api/v1/audit`	200	Sí
GET	`/api/v1/export`	200 (NDJSON)	Sí
GET	`/metrics`	200 (texto)	Sí

Códigos de error

Cuerpo estable: `{"error": "...", "code": "...", "retry_after_seconds": N?}`.

`code`	HTTP	Cuándo	Cabecera adicional
`bad_request`	400	Protocolo no admitido, muestra inválida, ventana temporal fuera de rango, filtro ilegible, actor vacío o demasiado largo, límites de superficie superados	—
`unauthorized`	401	Falta el token o no coincide	—
`not_found`	404	Activo o incidente inexistente	—
`rate_limited`	429	La cubeta de tokens de esa dirección está vacía	`Retry-After`
`locked_out`	429	La dirección cumple un bloqueo por fallos de autenticación	`Retry-After`
`internal`	500	Fallo inesperado; la causa se registra y **no** se devuelve	—

`Retry-After` nunca vale `0`: se eleva a `1` como mínimo.

Fuera de `ApiError`, las capas de `tower-http` producen `408 Request Timeout` (30 s) y `413 Payload Too Large` (por encima de `--max-body-kib`).

Métricas de Prometheus

Todas de tipo `gauge`, expuestas en `GET /metrics` (`text/plain; version=0.0.4`).

Métrica	Significado
`rootcause_uptime_seconds`	Segundos desde el arranque
`rootcause_assets_total`	Activos registrados
`rootcause_assets_online`	Activos que reportaron hace poco
`rootcause_incidents_open`	Incidentes sin resolver
`rootcause_incidents_critical`	Incidentes críticos sin resolver
`rootcause_exposed_services`	Servicios alcanzables fuera de su host
`rootcause_blocked_sources`	Direcciones bloqueadas por el plano de control
`rootcause_posture_score`	Puntuación de postura, 0 a 100
`rootcause_perimeter_locked_sources`	Direcciones cumpliendo un bloqueo ahora mismo
`rootcause_perimeter_tracked_clients`	Direcciones vigiladas por el perímetro
`rootcause_incidents_by_category{category="..."}`	Incidentes abiertos por categoría

Estados

Máquina	Estados	Transiciones
Incidente	`open` → `acknowledged` → `resolved`	Manual vía `POST /api/v1/incidents/{id}/status`; automática a `resolved` por `auto_resolve`; **vuelve a** `open` si la condición reaparece
Activo	`online` → `stale` → `offline`	Derivado de la antigüedad del último latido: 120 s y 600 s
Dirección en el perímetro	normal → limitada → bloqueada → normal	Cubeta vacía; umbral de fallos; expiración del bloqueo

Eventos registrados

Acciones que aparecen en `audit_log`:

`action`	`actor`	Cuándo
`incident.auto_resolved`	`rootcause-engine`	La condición dejó de observarse en una superficie inspeccionada
`incident.status_changed`	El `actor` de la petición (por omisión `console-user`)	Cambio manual de estado

Razones registradas en `defense_events`:

`reason`	Significado
`rate.limit`	Petición rechazada por la cubeta de tokens
`auth.lockout`	La dirección alcanzó el umbral de fallos y quedó bloqueada
`auth.rejected`	Token ausente o incorrecto

Archivos de configuración

Archivo	Formato	Consumidor	Obligatorio
`rust-toolchain.toml`	TOML	`rustup`	Sí, para compilar
`Cargo.toml` / `Cargo.lock`	TOML	`cargo`	Sí
`rustfmt.toml`	TOML	`cargo fmt`	No
`deny.toml`	TOML	`cargo deny`	Solo en CI
`.cargo/audit.toml`	TOML	`cargo audit`	Solo en CI
`.gitleaks.toml`	TOML	`gitleaks`	Solo en CI
`.hadolint.yaml`	YAML	`hadolint`	Solo en CI
`.markdownlint-cli2.jsonc`	JSONC	`markdownlint-cli2`	Solo en CI
`.env.example`	Texto	Persona (plantilla)	No
`compose.yml`	YAML	`docker compose`	Solo para contenedor
Política de detección	JSON	`--policy-file`	No; hay valores por omisión auditados

Anterior: 04 · Mapa completo del código · **Siguiente:** 06 · Explicación profunda del código · **Índice:** README